Task Management API

This project is a task management API built with Express, JWT authentication, role-based access control, and optional PostgreSQL persistence. It supports a local in-memory mode for quick testing and an optional database-backed mode for more realistic app behavior.

Architecture Overview



```
                                    | - tasks                      |
                                    +------------------------------+
```

Request flow:

1. A client sends an HTTP request to the API.
2. Express loads middleware and validates the request.
3. The matching route handler invokes the relevant controller.
4. The controller reads or updates data from either PostgreSQL or the in-memory fallback store.
5. The app returns JSON responses and central error handling catches invalid requests.

Features

- User registration and login
- JWT-based authentication
- Role-based access for admin and standard users
- Task CRUD operations
- Ownership checks on user-specific task access
- Admin-only task and user management routes
- Filtering by status and priority
- Sorting by created date or due date
- Search by task title/description
- Optional database mode using PostgreSQL and Sequelize
- Seeded starter roles, users, and tasks when the database is available
- Secure HTTP response headers using Helmet
- Centralized not-found and error handling middleware
- In-memory fallback for local development when no database is configured

Prerequisites

Before running the app, make sure you have:

- Node.js installed
- npm installed
- PostgreSQL (optional, only needed if you want DB-backed storage)
- Postman installed (or Postman Desktop App)

Setup Instructions

1. Clone or open the project

Open the project folder in your terminal or VS Code workspace.

2. Install dependencies

```
npm install
```

3. Configure environment variables

Copy the example environment file and adjust values if needed:

```
copy .env.example .env
```

Example contents:

```
PORT=3001
JWT_SECRET=your-secret-key
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/task_management
POSTGRES_SSL=false
```

Notes:

- If `DATABASE_URL` is set and the database is reachable, the app connects to PostgreSQL.
- If `DATABASE_URL` is missing or invalid, the app automatically falls back to in-memory storage.
- For local development without a database, you can leave `DATABASE_URL` unset.

4. Start the API server

```
npm run start
```

The app runs locally on:

```
http://localhost:3001
```

5. Run the Postman collection

Option A: Run the collection manually in Postman

1. Open Postman.
2. Import `TaskManagementAPI.postman_collection.json`.
3. Execute the requests in order.

Option B: Run it automatically with npm

```
npm run test:postman
```

This command starts the API, runs the collection, and validates the endpoints.

6. Run the Mocha test suite

```
npm test
```

This executes the project-level test suite:

- API integration tests (`tests/api.test.js`) using Mocha + Supertest
- Database smoke test (`tests/database.test.js`)

Expected outcomes:

- If `DATABASE_URL` is valid and reachable, you should see all tests passing (for example, `5 passing`).
- If `DATABASE_URL` is missing or unavailable, the DB smoke test is skipped and Mocha reports one pending test.

7. Optional: validate the database connection

If you configured `DATABASE_URL`, run the smoke test:

```
npm run test:db
```

This checks connectivity and confirms the expected tables are present.

8. Security headers check

The app enables Helmet middleware for secure HTTP responses at startup. This adds headers such as `X-Frame-Options`, `X-Content-Type-Options`, and other baseline protections without disabling API functionality.

Seeded users

The app seeds starter data automatically when it starts. These accounts are available for testing:

- Admin
    - Username: `admin`
    - Password: `admin123`
- Author user
    - Username: `alice`
    - Password: `alice123`

When PostgreSQL is available, the seed routine populates the `roles`, `users`, and `tasks` tables. Otherwise, the app continues in the in-memory fallback mode.

Import the Postman collection

1. Open Postman.
2. Click the "Import" button in the top-left.

3. Select the file:
    - `TaskManagementAPI.postman_collection.json`
4. After import, the collection will appear in your Postman sidebar.

Using the collection

The collection includes requests for:

- Register a user
- Login as admin
- Login as alice
- View your profile
- View all users (admin)
- View your tasks
- Filter and search tasks
- Create a task
- Get a task by ID
- Update a task
- Delete a task
- Admin view all tasks
- Admin user management routes

Important

The collection stores the JWT token in a Postman variable called `authToken` automatically after login/register requests. Protected routes can then use that token without manual copying.

Base URL

The collection uses this default variable:

```
http://localhost:3001
```

If your server runs on a different port or host, update the `baseUrl` variable in Postman:

1. Select the collection in Postman.
2. Open the "Variables" tab.
3. Change `baseUrl` to the correct URL.

Example request flow

1. Run `Auth - Login as Admin`
2. Run `Get My Tasks`
3. Run `Get Tasks - Filtered Search`
4. Run `Create Task`
5. Run `Get Task by ID`
6. Run `Update Task`
7. Run `Delete Task`

Troubleshooting

Server does not start

Check that dependencies are installed:

```
npm install
```

Then start the app again:

```
npm run start
```

Database is not connecting

If `DATABASE_URL` is set but the app still reports in-memory mode, verify:

- the database is running
- credentials in `.env` are correct
- the database exists
- PostgreSQL accepts the connection on the configured port

Postman request fails with 401

Make sure you have successfully run a login or register request first so the `authToken` variable is populated.

Also verify that `JWT_SECRET` is set in `.env`. Token creation and token verification both use this value, so a missing/incorrect value can cause all protected routes to fail authentication.

Mocha shows one pending test

If `npm test` reports one pending test, it usually means the database smoke test was skipped because `DATABASE_URL` was not available or the database connection could not be established at test time.

Postman request fails with 403

This usually means the current logged-in user does not have permission for that route. Use the admin login for admin-only routes.

API endpoints

Authentication

- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/auth/me`
- `GET /api/auth/profile`
- `GET /api/auth/users`

Tasks

- `GET /api/tasks`
- `POST /api/tasks`
- `GET /api/tasks/:taskId`
- `PUT /api/tasks/:taskId`
- `DELETE /api/tasks/:taskId`
- Filter parameters: `status`, `priority`, `sortBy`, `order`, `search`

Admin

- `GET /api/admin/tasks`
- `DELETE /api/admin/tasks/:taskId`
- `GET /api/admin/users`
- `DELETE /api/admin/users/:userId`

Notes

- The app defaults to in-memory storage when no valid database configuration is present.
- When `DATABASE_URL` is configured, the app uses Sequelize and PostgreSQL.
- The project includes a Postman collection and a database smoke test to support local verification.
- Data resets on restart when the app is running in in-memory mode.